

Path	Type	Summary (Main Role)	API Endpoi
<code>/admin/index.html</code>	HTML (Admin UI)	Main admin single-page interface for QualiJournal. Provides the dashboard for editors to review, approve, and publish news. Contains embedded JS for UI logic (buttons for data refresh, flows, report generation, etc.) ¹ ² . Shows KPI stats and lists collected articles.	(n/a – no endpoints defined in H
<code>/admin/index_lite_black.html</code>	HTML (Alt. UI theme)	A minimal “Lite Black” themed admin UI page (dark-mode focused) provided as a fallback or older version. Contains basic admin interface similar to main index but with simplified style. FastAPI serves this if the main <code>index.html</code> is missing ⁶ .	(n/a)
<code>/admin/db.py</code>	Python (DB utility)	Database connector for admin back-end. Defines <code>make_engine</code> (likely using SQLAlchemy) to connect to the database (e.g. Cloud SQL Postgres in cloud mode) ⁸ . Used to enable persistent storage of data (when <code>MODE</code> is cloud). If unavailable, the server runs in no-DB “local” mode ⁹ .	(n/a)

/server_quali.py

Python (FastAPI
back-end)

Main server application for QualiJournal Admin. Sets up the FastAPI app and all API endpoints ¹⁰. Includes routes for health, config, authentication, and core admin operations: **Content flows** (“Community/Keyword/Daily” collection and publish flows) ¹¹, **async task management** with Server-Sent Events (SSE) for long-running jobs ¹², **report generation & export** (/api/report , /api/export/{md|csv}) ¹², **content enrichment** (summary/translation) endpoints, **log viewing** (/api/logs/*) ¹³, etc. It also serves the admin UI (returns index.html on GET / ⁶) and mounts static file directories for reports and enriched content ¹⁴. Implements admin token auth (via header or Bearer token) for protected routes ¹⁵ ¹⁶.

Numerous endpoints (/api/*): **Flows:** /api/flow/community/api/flow/daily , /api/flow/keyword to trigger orchestrator flows ¹⁷ ¹⁸. **Tasks:** /api/tasks/flow (launch async job) and /api/tasks/{job_id}/stream (SSE stream of logs) ¹⁹, /api/tasks/{job_id}/cancel ²⁰. **Report/Export:** POST /api/report to generate data report ²¹, GET /api/export/{md|csv} to download report ²²; **Enrichment:** POST /api/enrich/keyword & api/enrich/selection/summarization/translation. **Tools:** POST /api/tools/repair and /api/tools/approve_t/run_maintenance_scripts ²⁴. **Publishing:**

Path	Type	Summary (Main Role)	API Endpoints
			api/items {id}/publ (approve & publish one 26 , /api. publish- keyword (finalize key edition) 27 Community api/ community save (save curated community 28 ; Config Gate: PATC api/confi gate_requ (set approva gate count) Logs: GET api/logs logs) and / logs/{nam (view or download lo 30 31 ; etc (Additional GET /heal and some d test endpoi like /api/ *.)

Path	Type	Summary (Main Role)	API Endpoi
<code>/orchestrator.py</code>	Python (CLI orchestrator)	Orchestrator script that handles data collection and publication workflows. Runs outside the web server (invoked via subprocess by <code>server_quali</code>). Accepts command-line flags to perform various tasks in the content pipeline: e.g. <code>--collect-community</code> fetches community posts (likely via Reddit API/PRAW) and saves them (to <code>selected_community.json</code>), <code>--collect-keyword <kw></code> gathers news articles for a given keyword (using sources in <code>smt_sources.json</code> & <code>supply_sources.json</code> ³⁸), <code>--approve-keyword-top 20</code> auto-approves top 20 articles, <code>--publish-community</code> or <code>--publish-keyword <kw></code> generates final outputs (markdown/CSV reports) for community or keyword edition, etc. These flags are used in combination for “daily” or “keyword” workflows ¹⁷ ³⁹. The orchestrator reads configuration from <code>config.json</code> (scoring thresholds, etc.) and uses external source lists (feeds). It produces JSON files like <code>selected_keyword_articles.json</code> (work-in-progress selection) and updates <code>selected_articles.json</code> (final selection) as well as archived report files.	(No direct A invoked via flags throug server calls)
<code>/auth_utils.py</code>	Python (Auth helper)	(Optional) Authentication utility for JWT. Intended to verify JWT tokens for requests (e.g. Google Cloud Run identity tokens). Provides <code>verify_jwt_token</code> function to decode/validate tokens. The server tries to import this; if unavailable, it falls back to a no-op stub ⁴¹. In practice, this module might integrate with Google auth libraries to check OAuth ID tokens.	(n/a)

Path	Type	Summary (Main Role)	API Endpoi
<code>/logging_setup.py</code>	Python (Logging config)	(Optional) Logging configuration. Sets up a file logger with a specific format and output file (<code>logs/server.log</code>). The server imports this to initialize logging ³⁷ . Likely configures log rotation or custom formatting via <code>setup_logger(name, path)</code> . If this module is absent, the server falls back to basic logging and ensures the log directory exists ⁴³ .	(n/a)
<code>/qj_paths.py</code> (not present)	Python (Path helper)	Utility that was intended to provide a project-relative path function (<code>qj_paths.rel</code>). Not actually included in the repository, but several tools attempt to import it ⁴⁴ . Those tools fall back to defining <code>qj_rel</code> locally if import fails. Essentially, it would have centralized the logic for constructing file paths relative to the project root.	n/a
<code>/feeds/</code>	Directory (Feeds data)	Feeds directory containing external source definitions for news collection. Likely holds JSON files that list sources for community and official news. For example, <code>community_sources.json</code> resides here ⁴⁵ (list of community forums or subreddits to crawl), possibly along with other source files. This directory groups all external feed configurations and any related parsing logic (if any, though current implementation uses these definitions in the orchestrator).	(n/a)
<code>/feeds/community_sources.json</code>	JSON data (Community feeds)	JSON file listing community content sources (e.g. subreddits or forums) to pull articles from. Likely includes identifiers (subreddit names, etc.) and perhaps categories or weights. The orchestrator consults this file (or a root copy) when <code>--collect-community</code> is run ⁴⁵ . Ensures the system knows which community pages to scrape for relevant posts.	(n/a)

Path	Type	Summary (Main Role)	API Endpoi
<code>/smt_sources.json</code>	JSON data (News feeds)	JSON file listing “SMT” (likely Surface-Mount Technology / electronics manufacturing) news sources. Contains feed URLs or site info for official news related to electronics production. The orchestrator’s official collect (<code>collect_official</code> function) reads this to fetch the latest article from each source ³⁸ . Ensures industry-specific sources (e.g. IPC, I-Connect007, etc.) are included.	(n/a)
<code>/supply_sources.json</code>	JSON data (News feeds)	JSON file listing “Supply” news sources (likely related to supply chain or electronics supply industry). Similar structure to <code>smt_sources.json</code> , it enumerates URLs or feeds to collect news. The orchestrator uses this alongside SMT sources for official news collection ³⁸ . Provides coverage of supply chain news to complement technical standards news.	(n/a)
<code>/keywords.txt</code>	Text data	A text file listing keywords of interest (possibly one per line). May be used by the system to initiate “keyword news” editions – e.g. an editor selects a keyword from this list or the orchestrator uses these as defaults if none provided. Ensures consistency in keyword terminology.	(n/a)
<code>/keyword_synonyms.json</code>	JSON data	JSON mapping of keyword synonyms. Used to broaden the search/selection of articles: if <code>use_keyword_synonyms</code> is true in config ⁴⁷ , the orchestrator will treat listed synonyms as equivalent to the main keywords when filtering content. For example, it can ensure articles containing synonyms of the chosen keyword are not missed.	(n/a)

Path	Type	Summary (Main Role)	API Endpoi
/ selected_keyword_articles.json	JSON data (Working set)	Work-in-progress selection of articles (JSON). This file is the current list of collected articles (from both community and official sources) for the active keyword or daily edition. It includes metadata for each article (title, source, URL, summary, scores like <code>kw_hits</code>, <code>upvotes</code>, etc., and flags like <code>approved</code>, <code>state</code>) ⁴⁸ ⁴⁹. Initially populated by orchestrator (<code>--collect-*</code> commands) and then updated as editors approve or reject items. Serves as the source of truth for what's in the "Ready" list in the admin UI.	(n/a)
/selected_articles.json	JSON data (Final selection)	Final approved articles JSON. This file contains only the articles that have been approved for publication, in a similar structure to the above (with <code>approved</code> true, and usually <code>state: "ready"</code> or <code>"published"</code> for all) ⁵² ⁵³. It is generated by filtering the working set or by running the sync script, and represents the content that will actually be published/exported. The orchestrator uses this to produce the Markdown/CSV reports.	(n/a)

Path	Type	Summary (Main Role)	API Endpoi
<code>/selected_community.json</code>	JSON data (Community snapshot)	A JSON file holding collected community posts (and their metadata) from Reddit/forums. Created by <code>--collect-community</code> . Contains posts (with title, score, upvotes, etc.) relevant to the current date/keyword. Not always present in root; it might be saved under <code>archive/</code> after processing. The server references both locations ⁵⁶ . It's used to calculate community metrics and to merge community content into final outputs.	n/a
<code>/tools/</code>	Directory (Utility scripts)	Utility scripts for maintenance and batch operations. This folder contains various Python scripts used for managing or repairing data outside the main app. These include tools for auto-approving content, syncing files, verifying documentation, etc. They are not part of the API per se, but some can be triggered via special admin API calls (under <code>/api/tools/*</code>). Keeping them in <code>tools</code> separates these auxiliary functions from core logic.	(n/a)
<code>/tools/force_approve_top20.py</code>	Python script	Auto-approve top N articles. Scans the current selection JSON for articles, computes a score for each (combining factors like base score, upvotes, views, keyword hits, etc. with specific weights ⁵⁸), and marks the top N as approved ⁵⁹ . It sets their state to “ready” and date to today, then saves back to <code>selected_keyword_articles.json</code> . If not in dry-run mode, it then calls the sync script to update the final selection ⁶⁰ . This automates the editor’s approval process for the highest-ranked items. Default N=20 (thus “Top20”).	(no direct endpoint, b invoked by POST /api/t approve_to

Path	Type	Summary (Main Role)	API Endpoi
/tools/ sync_selected_for_publish.py	Python script	Sync selection to publish. Reads the work selection (<code>selected_keyword_articles.json</code>), filters only those with <code>approved=True</code>, normalizes their state to “ready”, and writes out the final selection JSON (<code>selected_articles.json</code>) ⁵² ⁵³. It ensures both the root and <code>/data</code> copies of <code>selected_articles.json</code> are updated, and prints the count of approved items. Essentially, this creates the publishable set from the working set. It’s used after approvals to prepare for report generation.	(no direct endpoint; c by other scr or manually
/tools/ repair_selection_files.py	Python script	Repair and normalize JSON files. Goes through the selection JSON files to fix any structural or data inconsistencies. It loads the working selection (<code>selected_keyword_articles.json</code> or its variants), and the final selection, and ensures: (1) Both files have the correct structure (date, items, articles) with <code>items</code> and <code>articles</code> pointing to the same list ⁶². (2) All articles have essential fields (adds default <code>title</code>, <code>source</code>, <code>date</code> if missing) and normalized values (e.g. ensures <code>approved</code> is boolean, and sets <code>state</code> to “ready” if approved or “candidate” if not) ⁶³. (3) Removes duplicate entries (by unique URL/title key) ⁶⁴. It then saves the cleaned working file and rebuilds the final selection with only approved items (essentially performing a sync as well) ⁶⁵. Useful if the JSON became corrupted or inconsistent (perhaps due to crashes or manual edits).	(no direct endpoint; b POST /api/t repair trigger ²⁴

Path	Type	Summary (Main Role)	API Endpoi
<code>/tools/find_anchor.py</code>	Python script	Anchor verification tool. Loads the project's markdown manifest ("MANIFEST-SSOT.md") which contains a table of files and expected anchor strings for each. For each file listed, it checks that the file exists and that all specified anchor substrings are present in that file's content ⁶⁷ . If any anchor is missing (or file is missing), it reports an error (exit code 1). This is used to ensure that certain key comments or markers (anchors) remain in the code – a way to prevent accidental deletion of important code sections or to ensure consistency (Single Source of Truth compliance).	(n/a)
<code>/tools/verify_manifest.py</code>	Python script	Inventory verification tool. Checks the JSON inventory file in <code>/_inventory/</code> to ensure it's properly formatted and complete. It loads <code>_inventory/inventory.json</code> and verifies that it contains an <code>items</code> list with each entry having at least a <code>path</code> and <code>sha256</code> hash ⁶⁸ . Essentially, it ensures the inventory (a machine-readable list of files and their hashes) is not empty and has the expected fields. This helps catch mistakes if the inventory wasn't updated or got corrupted.	(n/a)
<code>/_inventory/inventory.json</code>	JSON data (File inventory)	Inventory of all files in the project with their checksums. Each entry typically includes the file path, its size, and a SHA-256 hash, plus possibly metadata like component and anchor list ⁶⁹ . This is the Single Source of Truth (SSOT) for the project's contents, used to verify that documentation and code are in sync and to detect any unintended changes. By comparing hashes, one can ensure no files were altered unexpectedly.	(n/a)

Path	Type	Summary (Main Role)	API Endpoi
<code>/docs/</code>	Directory (Documentation)	Project documentation files. Contains Markdown documents for guidelines, checklists, and design notes. For example: <code>TAGGING.md</code> outlines code annotation standards (component, feature, anchor tags) ⁷⁰ , and <code>CHAT_START_CHECKLIST.md</code> may cover pre-release checks. These docs help developers and operators understand and maintain the system. They are not served by the application but reside in the repository for reference.	(n/a)
<code>/Dockerfile</code>	Config (Container build)	Docker container definition. Specifies how to build the QualiJournal Admin service image. Uses Python 3.11 slim base, sets environment (disables .pyc and enables unbuffered logging) ⁷¹ , installs Python dependencies from <code>requirements.txt</code> ⁷² , and then copies the project files into the image ⁷³ . Finally, it sets the entrypoint to run the FastAPI app with Uvicorn (listening on <code>\$PORT</code>) ⁷⁴ . This file ensures the app can be containerized for deployment (e.g. to Cloud Run).	(n/a)
<code>/.github/workflows/deploy.yml</code>	CI Workflow (GitHub Actions)	Continuous Deployment pipeline for the project. This GitHub Actions workflow triggers on pushes to the main branch. It authenticates to Google Cloud and builds/deploys the app to Cloud Run ⁷⁵ ⁷⁶ . It uses the Dockerfile (Cloud Run Source Deploy builds the image from source) and defines environment variables like project ID, region, service name, and the entrypoint command ⁷⁵ ⁷⁷ . The workflow ensures that every commit to main gets automatically released to the production service (after passing any checks). It also includes a no-op for pull requests (to not deploy from PRs) ⁷⁸ .	(n/a)

1 2 3 4 5 7 19 21 22 23 29 32 33 34 35 46 **index.html**
<https://github.com/cdmacs1003-cyber/quali-journal/blob/d530434e906709482cfa2ae5d8313b825f58ea30/index.html>

6 8 9 10 11 12 13 14 15 16 17 18 20 24 25 26 27 28 30 31 36 37 39 41 42 43 50 51 56
 57 61 **server_quali.py**
https://github.com/cdmacs1003-cyber/quali-journal/blob/d530434e906709482cfa2ae5d8313b825f58ea30/server_quali.py

38 40 **1003_1A퀄리 뉴스 제안.pdf**
<file:///file-9ckyN6UsWeG3crtkDBreya>

44 58 59 60 **force_approve_top20.py**
https://github.com/cdmacs1003-cyber/quali-journal/blob/d530434e906709482cfa2ae5d8313b825f58ea30/force_approve_top20.py

45 47 **config.json**
<https://github.com/cdmacs1003-cyber/quali-journal/blob/d530434e906709482cfa2ae5d8313b825f58ea30/config.json>

48 49 **selected_articles.json**
https://github.com/cdmacs1003-cyber/quali-journal/blob/d530434e906709482cfa2ae5d8313b825f58ea30/selected_articles.json

52 53 54 55 **sync_selected_for_publish.py**
https://github.com/cdmacs1003-cyber/quali-journal/blob/d530434e906709482cfa2ae5d8313b825f58ea30/sync_selected_for_publish.py

62 63 64 65 66 **repair_selection_files.py**
https://github.com/cdmacs1003-cyber/quali-journal/blob/d530434e906709482cfa2ae5d8313b825f58ea30/repair_selection_files.py

67 **find_anchor.py**
https://github.com/cdmacs1003-cyber/quali-journal/blob/d530434e906709482cfa2ae5d8313b825f58ea30/tools/find_anchor.py

68 **verify_manifest.py**
https://github.com/cdmacs1003-cyber/quali-journal/blob/d530434e906709482cfa2ae5d8313b825f58ea30/tools/verify_manifest.py

69 70 **TAGGING.md**
<https://github.com/cdmacs1003-cyber/quali-journal/blob/d530434e906709482cfa2ae5d8313b825f58ea30/docs/TAGGING.md>

71 72 73 74 **Dockerfile**
<https://github.com/cdmacs1003-cyber/quali-journal/blob/d530434e906709482cfa2ae5d8313b825f58ea30/Dockerfile>

75 76 77 78 **deploy.yml**
<https://github.com/cdmacs1003-cyber/quali-journal/blob/d530434e906709482cfa2ae5d8313b825f58ea30/deploy.yml>